计划任务书

---

1. 引言

1.1 编写目的

本文档将《可行性研究》的决策依据与《项目开发计划》的执行方案高度集成，旨在为“做菜模拟器”项目提供从技术架构到人员调度的全方位指导。本文档预期的读者包括课程指导教师、项目开发组全体成员。

1.2 背景

- **软件名称**：做菜模拟器 (Cooking Simulator)
- **开发者**：b2606小组
- **用户定位**：烹饪初学者、家庭厨艺练习者、对数字化烹饪感兴趣的年轻人。

1.3 参考资料

- GB8567-88《计算机软件要求说明》
- 《软件工程导论》
- OpenAI/DeepSeek API 技术开发文档

---

2. 系统概述与技术可行性

2.1 核心业务逻辑与处理流程

系统的核心在于将“非线性的烹饪动作”**转化为**“线性的结构化日志”。

1. **驱动层（数据输入）**：系统读取结构化的“数字菜谱” JSON 文件，定义该菜品的“标准动作链”（如：第120秒，加入食盐5g）。
2. **交互层（实时采集）**：玩家在模拟环境中的每一次有效交互（点击、拖拽、加热开关）都会被“动作监听器”捕获。
3. **处理层（日志生成）**：系统实时封装 `(Action_ID, Action_Name, Precise_Timestamp, Value)`。例如：`(A005, Add_Salt, 02:05.44, 5g)`。
4. **模式设计**：
   - **学习模式**：菜谱始终可以点击查阅，计时器和操作提示并行展示。
   - **挑战模式**：菜谱不再实时显示，不再提供操作提示，玩家需要记忆操作节奏和时间节点，系统仅记录玩家行为并在结束后评分。
5. **评价层（AI 决策）**：在挑战结束时，系统将“标准动作链”与“玩家动作链”进行差分对比，并附带 Prompt（提示词）通过 API 发送至大模型。

2.2 技术栈选择深度论证

2.2.1 跨平台框架：Unity 3D

- **选型理由**：
  - **跨平台一致性**：本项目要求全平台适配（PC/Web/Mobile）。Unity 的跨平台底层抽象可以确保同一套 UI 逻辑和计时算法在 Windows 和移动端上表现一致。
  - **异步处理能力**：Unity 的协程（Coroutine）和异步编程模型（UniTask）非常适合处理 LLM API 的网络延迟，确保在等待 AI 评价时，前端界面不会假死。
  - **UI 扩展性**：UGUI 系统可以轻松实现学习模式中要求的悬浮指引窗口，并支持多分辨率下的 UI 自适应。
- **可行性论证**：开发团队已具备 Unity 基础，且 Unity 官方对 WebGL 的支持已趋于成熟，能够解决网页版模拟器的资源加载问题。

2.2.2 数据交换格式：JSON (JavaScript Object Notation)

- **选型理由**：
  - **语义对齐**：JSON 的键值对结构天然符合“属性-数值”的菜谱特征。
  - **AI 友好性**：当前主流大模型（如 GPT-4, Qwen, DeepSeek）在训练阶段接触了海量的 JSON 数据，能够非常精准地识别结构化 Log 中的逻辑错误（如“时间点超限”或“操作顺序颠倒”）。
  - **序列化开销低**：JSON 文本轻量，即便在高频记录操作日志时，也不会占用过多 CPU 和内存资源。

2.2.3 AI 逻辑判定：LLM API

- **选型理由**：
  - **解决非确定性判定**：传统编程难以通过 `if-else` 判断“晚放盐 10 秒”对菜品口味的具体影响。而 LLM 可以基于其庞大的语料库进行**语义推理**，给出人性化的描述性评价。
  - **动态容差算法**：利用 AI 的零样本推理（Zero-shot Reasoning），系统可以设置一个灵活的评分标准（如：关键步骤 2 秒容差，非关键步骤 10 秒容差），AI 能够理解这种权重差异。
- **可行性论证**：目前主流 API 的 Token 价格已大幅下降，对于单次烹饪挑战产生的日志（约 1k-2k Tokens），成本可控。

2.3 关键技术难点与可行性对策

为了体现报告的严谨性，必须加入对**技术难点**的预判：

1. **大模型prompt预设**：
   - **风险**：对于运行过程中的日志信息，大模型可能不能输出合适格式的报告。
   - **对策**：做一些提示词工程工作，写md文档规定输出。
2. **API 响应延迟与用户体验**：
   - **风险**：AI 评分可能需要 3-5 秒，用户等待体验不佳。

- **对策**：设计“大厨点评中...”的过渡动画，并采用流式传输（Streaming）技术，让 AI 的文字逐字跳出，增加交互仪式感。

3. **菜谱数据的可扩展性**：

- **风险**：后期增加新菜谱是否需要重新写代码？
- **对策**：实现“菜谱解析器”模块。只要按照标准 JSON 格式编写新文件并存入 `StreamingAssets` 文件夹，系统即可自动加载并生成新的关卡。

4. **苹果端适配**：

- **风险**：apple 和 android 操作系统的差异性导致适配困难
- **对策**：依托 Auto Layout 弹性布局 + 安全区适配 + 多分辨率资源兼容，同时兼顾 iOS 版本、深浅色模式及灵动岛 / 刘海等异形屏特性做统一兼容适配

2.4 可行性分析结论

通过对开发工具链（Unity）、数据结构（JSON）及判定算法（AI）的综合论证，本项目在**技术实现上无盲点**，在**硬件性能上无压力**，在**成本投入上可覆盖。**

结论：本项目技术方案可行，可以进入开发实施阶段。

---

3. 工作分解结构 (WBS) 与详细分工

这是项目的核心执行部分。我们将任务分为五个主要工作包（Work Package）：

3.1 任务分工表

任务 ID	任务类别	任务名称	详细内容描述	负责人
WP1	**策划分析**	**菜谱标准化与Prompt设计**	定义菜谱 JSON 标准（含调料、主料、时点、动作与容差）；设计并调优 AI 提示词，确保评分在 0-100 之间波动。	项目经理/策划
WP2	**前端开发**	**跨平台 UI 系统**	开发主界面、学习/挑战模式切换逻辑；实现多端分辨率自适应；编写悬浮窗菜谱指引组件。	前端/UI 开发
WP3	**核心引擎**	**计时交互与日志引擎**	编写全局计时器模块；实现交互物触发记录器；实时将用户操作序列化为 JSON Log。	核心引擎开发
WP4	**后端集成**	**API 通讯与反馈解析**	封装 C# HTTP 请求模块；处理 API 异步回调逻辑；解析 AI 返回的描述文本并渲染至评分结算页。	AI/后端开发
WP5	**测试交付**	**兼容性测试与数据导入**	导入首批 20 道菜谱；在 Web、手机、PC 三端执行功能回归测试；编写用户手册。	测试/文档

3.2 人员组织与安排

角色	核心职责
项目经理 / 组长	整体计划制定、进度管控、任务分配、与课程组沟通协调
产品 / 策划	菜谱标准化制定、学习 / 挑战模式规则设计、评分机制与验收标准定义
前端 / UI 开发	跨平台界面实现、悬浮提示开发、计时器显示、交互反馈效果实现
核心引擎开发	计时逻辑开发、交互事件捕获、日志结构化处理、学习 / 挑战模式逻辑判定
AI / 后端开发	日志与菜谱比对、AI API 封装、异步请求处理、返回结果解析、评分汇总
测试 / 文档人员	功能与兼容性测试、Bug 记录跟踪、用户手册编写、验收文档整理

---

4. 进度计划 (里程碑)

项目计划周期为 8 周，采用瀑布模型与迭代模型结合的方式：

- **M1 (第 1-2 周)：需求锁定与技术原型**
  - 完成《任务计划书》与《需求规格说明书》。
  - 输出：菜谱 JSON 格式草案、AI Prompt 测试报告。
- **M2 (第 3-5 周)：核心功能迭代**
  - 完成计时引擎与交互监听模块。
  - 输出：可在 PC 端运行的 Cooking Demo。
- **M3 (第 6-7 周)：AI 联调与多端适配**
  - 完成 AI 接口对接，实现“操作-评价”完整闭环。
  - 输出：Android APK 与 WebGL 发布版。
- **M4 (第 8 周)：交付与验收**
  - 完成全部 Bug 修复，提交结项报告。

4.1 详细周计划

周数	核心工作内容
第 1 周	明确学习 / 挑战模式功能，确定菜谱数据结构，编写首版菜谱 JSON 模板
第 2 周	完成菜谱标准与 AI 评分 Prompt 设计，通过评审后确定学习模式 UI 布局
第 3 周	开发全局计时器、操作日志记录模块，实现基本交互事件捕获
第 4 周	实现学习模式场景，完成悬浮菜谱提示窗口、时间轴及操作反馈逻辑
第 5 周	开发挑战模式逻辑，关闭实时菜谱提示，完成时序判定与日志采集
第 6 周	集成 AI 评价模块，实现日志与菜谱比对、评分文字反馈及总分计算

周数	核心工作内容
第 7 周	完成多端适配与回归测试，修复兼容性问题，优化交互体验
第 8 周	整理交付材料，完成最终项目演示与验收

---

5. 交付产品与验收标准

5.1 软件产品

- **核心交付包**：`.apk` (Android，核心)、`index.html` (Web，核心)；
- **补充交付包**：`.exe` (Windows，轻量化适配，非核心)；
- **源码**：包含 Unity 工程文件、C# 脚本、JSON 菜谱库（需标注移动端/PC 端适配相关代码区分）。

5.2 文档产品

- 《需求规格说明书》（补充玩法体验类需求定义）；
- 《软件概要设计说明书》（明确移动端为核心适配端、PC 端轻量化适配方案）；
- 《测试分析报告》（分端标注测试优先级与达标标准）；
- 《用户手册》（分移动端/PC 端说明操作方式）。

5.3 验收量化指标

5.3.1 核心指标

1. **逻辑准确性**：计时器误差 < 100ms；玩家所有有效操作（点击/拖拽/加热开关）日志捕捉率 100%；
2. **玩法体验**：
   - 学习模式：悬浮菜谱指引无遮挡，操作提示触发率 100%，反馈延迟 < 200ms；
   - 挑战模式：无菜谱/提示泄露，操作时序错误可被精准记录；
3. **AI 评价效果**：
   - 对 >5 秒时间误差的敏感度 ≥90%；
   - AI 评价描述与操作错误匹配率 ≥95%；
   - 不同操作误差下的评分差值 ≥10 分（满分 100）；
4. **兼容性**：Chrome（≥90 版本）、主流安卓手机（Android 10+）可正常开启并完成全流程烹饪。

5.3.2 可选指标

1. **PC 端适配**：Win11 系统下可启动游戏，完成一次完整的学习/挑战模式流程，无核心功能崩溃；
2. **PC 端界面**：16:9 窗口化显示无严重布局错乱。

---

6. 风险评估与对策

- **技术风险**：LLM API 响应时间受网络波动影响。
  - *对策*：引入异步加载机制，在结算阶段展示“评价生成中...”动画。
- **开发风险**：全平台适配工作量超出预期。
  - *对策*：优先在pc和web端实现满足核心玩法的mvp产品，再酌情考虑多移动端适配。

---

7. 预算与资源支持

- **设备支持**：PC 开发机 1 台，Android 测试机 1 台，苹果测试机一台，鸿蒙测试机一台。
- **软件支持**：Unity 编辑器、Git 版本控制、Postman。
- **经费预算**：预留 150-300 元用于 AI API 的 Token 消耗及 Web 托管空间费用。